



Departamento de Lenguajes y
Sistemas Informáticos

Consumo de Servicios RESTful

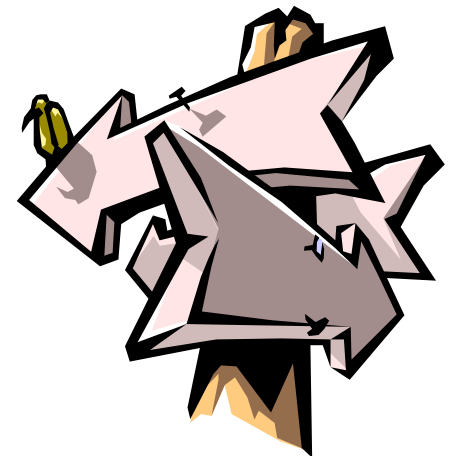
Integración



Arquitectura e Integración del Software

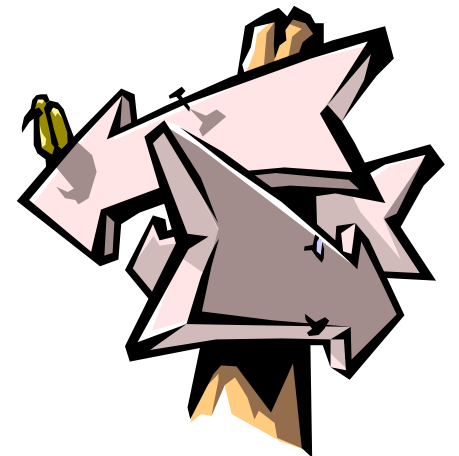
Índice

- Introducción
- Mapeo de datos JSON
- Consumo de servicios RESTful con Spring



Índice

- **Introducción**
- Mapeo de datos JSON
- Consumo de servicios RESTful con Spring





RESTful APIs

HTTP

+



CREATE



READ



UPDATE



DELETE

C

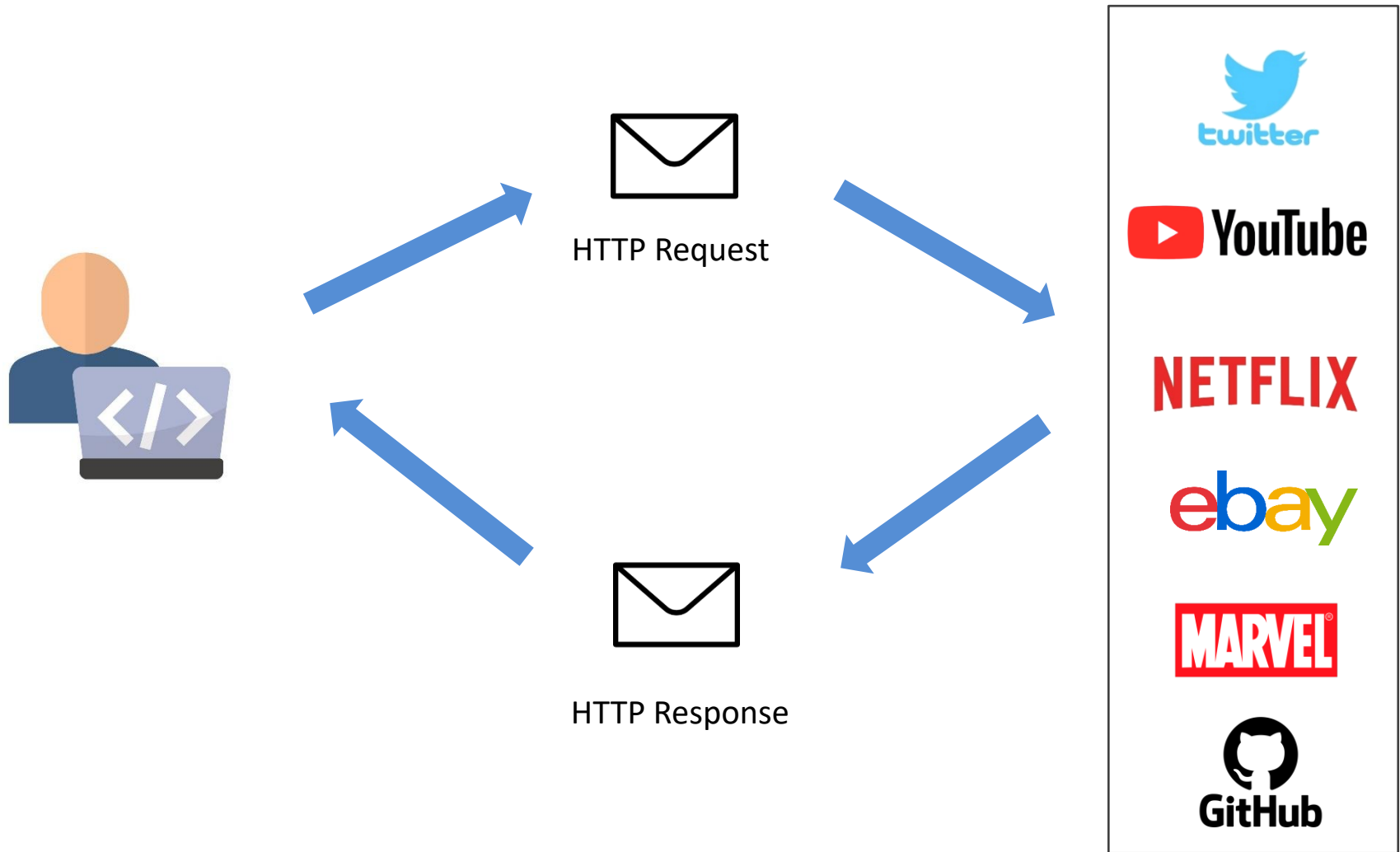
R

U

D

RESOURCES

Introducción



Introducción



<https://api.restful-api.dev/objects/1>



Parameter *objectID*

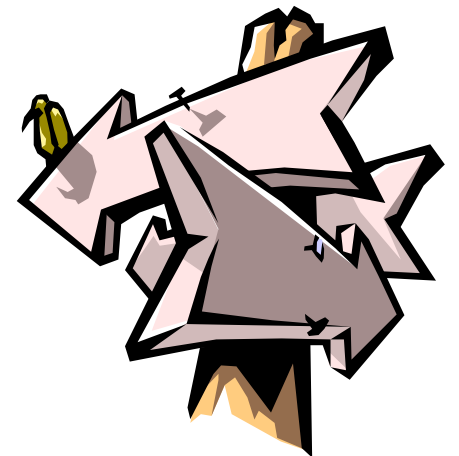


```
{
  "id": "1",
  "name": "Google Pixel 6 Pro",
  "data": {
    "color": "Cloudy White",
    "capacity": "128 GB"
  }
}
```



Índice

- Introducción
- **Mapeo de datos JSON**
- Consumo de servicios RESTful con Spring



Mapecto de datos JSON

¿Cómo procesar los datos en formato JSON empleados en la interacción con los servicios?



```
{  
  "id": "1",  
  "name": "Google Pixel 6 Pro",  
  "data": {  
    "color": "Cloudy White",  
    "capacity": "128 GB"  
  }  
}
```

Mapecto de datos JSON

¡Almacenemos los datos en objetos Java!



```
{  
  "id": "1",  
  "name": "Google Pixel 6 Pro",  
  "data": {  
    "color": "Cloudy White",  
    "capacity": "128 GB"  
  }  
}
```

Mapecto de datos JSON

```
package aiss2526.restclient.model.objects;  
  
> import ...  
  
@JsonIgnoreProperties(ignoreUnknown = true)  
public class ApiObject {  
  
    @JsonProperty("id") 4 usages  
    private String id;  
  
    @JsonProperty("name") 4 usages  
    private String name;  
  
    @JsonProperty("data") 4 usages  
    private Data data;  
}
```

```
{  
  "id": "1",  
  "name": "Google Pixel 6 Pro",  
  "data": {  
    "color": "Cloudy White",  
    "capacity": "128 GB"  
  }  
}
```

Plain Old Java Object (**POJO**)

Mapecto de datos JSON

```
package aiss2526.restclient.model.objects;  
  
> import ...  
  
@JsonIgnoreProperties(ignoreUnknown = true)  
public class ApiObject {  
  
    @JsonProperty("id") 4 usages  
    private String id;  
    @JsonProperty("name") 4 usages  
    private String name;  
    @JsonProperty("data") 4 usages  
    private Data data;  
}
```

```
{  
  "id": "1",  
  "name": "Google Pixel 6 Pro",  
  "data": {  
    "color": "Cloudy White",  
    "capacity": "128 GB"  
  }  
}
```

Plain Old Java Object (**POJO**)

Mapecto de datos JSON

```
package aiss2526.restclient.model.objects;  
  
> import ...  
  
@JsonIgnoreProperties(ignoreUnknown = true)  
public class ApiObject {  
  
    @JsonProperty("id") 4 usages  
    private String id;  
    @JsonProperty("name") 4 usages  
    private String name;  
    @JsonProperty("data") 4 usages  
    private Data data;  
}
```

```
{  
  "id": "1",  
  "name": "Google Pixel 6 Pro",  
  "data": {  
    "color": "Cloudy White",  
    "capacity": "128 GB"  
  }  
}
```

Plain Old Java Object (**POJO**)

Mapecto de datos JSON

jsonschema2pojo

 Star 5,907  Tweet

Generate Plain Old Java Objects from JSON or JSON-Schema.

```
1 {
2   "type": "object",
3   "properties": {
4     "foo": {
5       "type": "string"
6     },
7     "bar": {
8       "type": "integer"
9     },
10    "baz": {
11      "type": "boolean"
12    }
13  }
14 }
```

Package

Class name

Source type:
☐ JSON Schema ☒ JSON
☐ YAML Schema ☐ YAML

Annotation style:
☒ Jackson 2.x ☐ Gson
☐ Moshi ☐ None

☐ Generate builder methods
☐ Use primitive types
☐ Use long integers
☒ Use double numbers
☐ Use Joda dates
☒ Include getters and setters
☐ Include constructors
☐ Include `hashCode` and `equals`
☒ Include `toString`
☐ Include JSR-303 annotations
☐ Allow additional properties
☐ Make classes serializable
☐ Make classes parcelable
☐ Initialize collections

Property word delimiters:

Preview 

Zip 

<https://www.jsonschema2pojo.org/>

Maapeo de datos JSON

Librería Jackson - Anotaciones

- `@JsonProperty`. Permite indicar el nombre de la propiedad en JSON.

```
public class Name {  
    @JsonProperty("firstName")  
    public String _first_name;  
}
```

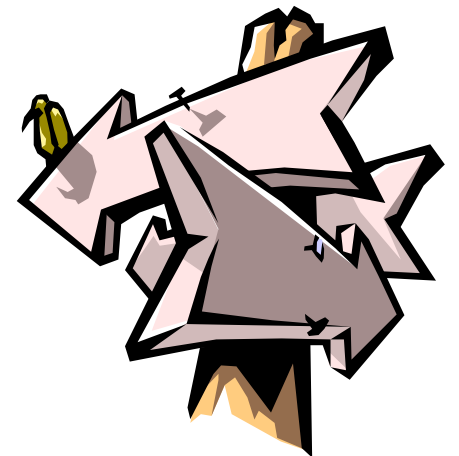
- `@JsonIgnoreProperties`. Permite ignorar una o más propiedades. También es posible ignorar todas las propiedades para las que no se han definido atributos Java (`ignoreUnknown=true`)

```
@JsonIgnoreProperties({ "extra", "uselessValue" })  
public class Value {  
    public int value;  
}
```

```
@JsonIgnoreProperties(ignoreUnknown=true)  
public class PojoWithAny {  
    public int value;  
}
```

Índice

- Introducción
- Mapeo de datos JSON
- **Consumo de servicios RESTful con Spring**



Consumo de servicios RESTful con Spring

Configuración

```
@SpringBootApplication
public class RestClientApplication {

    public static void main(String[] args) { SpringApplication.run(RestClientApplication.class, args); }

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) { return builder.build(); }

}
```

Consumo de servicios RESTful con Spring

Consulta - GET

- Opción 1. Usar el método *getForObject* para obtener el objeto del cuerpo de la respuesta.

```
User user = restTemplate.getForObject(uri, User.class);  
System.out.println("Hello " + user.getFirstName());
```

Para obtener un listado
(array) de objetos

```
Commit[] commits = restTemplate  
    .getForObject(baseUrl + owner + "/" + repo + "/commits", Commit[].class);  
  
return Arrays.stream(commits).toList();
```

Consumo de servicios RESTful con Spring

Consulta - GET

- Opción 2. Usar el método *getForEntity* para obtener el mensaje de respuesta completo, incluyendo cabeceras y cuerpo.

```
ResponseEntity<User> response = restTemplate.getForEntity(uri, User.class);  
System.out.println("Hello " + response.getBody().getFirstName());
```

Consumo de servicios RESTful con Spring

Consulta - GET

- Opción 3. Usar el método *exchange* para tener control total sobre los mensajes de petición y respuesta.

```
ResponseEntity<User> response = restTemplate.exchange(uri, HttpMethod.GET, null, User.class);  
System.out.println("Hello" + response.getBody().getFirstName());
```

```
// Create request  
HttpHeaders headers = new HttpHeaders();  
headers.set("Authorization", "Bearer " + token);  
HttpEntity<Commit[]> request = new HttpEntity<>(null, headers);  
  
// Send request  
ResponseEntity<Commit[]> response = restTemplate.exchange(uri, HttpMethod.GET, request, Commit[].class);
```



Mensaje HTTP de petición (necesario cuando se quieren enviar cabeceras)

Consumo de servicios RESTful con Spring

Creación - POST

- Opción 1. Usar el método *postForObject* para enviar el objeto directamente.

```
createdUser = restTemplate.postForObject(uri, user, User.class);
```

- Opción 2. Usar el método *postForEntity* para obtener el mensaje de respuesta completo, incluyendo cabeceras y cuerpo.

```
ResponseEntity<User> response = restTemplate.postForEntity(uri, user, User.class);  
createdUser = response.getBody();
```

- Opción 3. Usar el método *exchange* para tener control total sobre los mensajes de petición y respuesta.

```
HttpEntity<User> request = new HttpEntity<>(user);  
ResponseEntity<User> response = restTemplate.exchange(uri, HttpMethod.POST, request, User.class);  
createdUser = response.getBody();
```

Consumo de servicios RESTful con Spring

Actualizar - PUT

- Opción 1. Usar el método *put* para actualizar el objeto directamente.

```
restTemplate.put(uri, user, User.class);
```

- Opción 2. Usar el método *exchange* para tener control total sobre los mensajes de petición y respuesta.

```
HttpEntity<User> request = new HttpEntity<>(user);  
ResponseEntity<User> response = restTemplate.exchange(uri, HttpMethod.PUT, request, User.class);  
User updatedUser = response.getBody();
```

Consumo de servicios RESTful con Spring

Eliminar - DELETE

- Opción 1. Usar el método *delete* para actualizar el objeto directamente.

```
restTemplate.delete(uri);
```

- Opción 2. Usar el método *exchange* para tener control total sobre los mensajes de petición y respuesta.

```
ResponseEntity<Void> response = restTemplate.exchange(uri, HttpMethod.DELETE, null, Void.class);
```

Consumo de servicios RESTful con Spring

Tratamiento de errores

Por defecto, RestTemplate lanza las siguientes excepciones:

- **HttpClientErrorException**. Si el código HTTP de respuesta es 4xx (error de cliente).
- **HttpServerErrorException**. Si el código HTTP de respuesta es 5xx (error de servidor).
- **UnknownHttpStatusCodeException**. Si el código HTTP es desconocido o personalizado.

Todas estas excepciones son subclases de **RestClientResponseException**.

Enlaces

- Postman

<https://www.postman.com/>

- Spring Framework / Spring Boot

<https://www.baeldung.com/rest-template>

- Librería Jackson

<https://github.com/FasterXML/jackson>

- REST Template

<https://www.baeldung.com/rest-template>

https://www.tutorialspoint.com/spring_boot/spring_boot_rest_template.htm

<https://howtodoinjava.com/spring-boot2/resttemplate/spring-restful-client-resttemplate-example/>

- Tratamiento de errores

<https://www.baeldung.com/spring-rest-template-error-handling>

Disclaimer and Terms of Use

All material displayed on this presentation is for teaching and personal use only.

Many of the images that have been used in the presentation are Royalty Free images taken from <http://www.everystockphoto.com/>. Other images have been sourced directly from the Public domain, from where in most cases it is unclear whether copyright has been explicitly claimed. Our intention is not to infringe any artist's copyright, whether written or visual. We do not claim ownership of any image that has been freely obtained from the public domain. In the event that we have freely obtained an image or quotation that has been placed in the public domain and in doing so have inadvertently used a copyrighted image without the copyright holder's express permission we ask that the copyright holder writes to us directly, upon which we will contact the copyright holder to request full written permission to use the quote or images.